

You're speaking my language here—let's make this something you could walk through on a whiteboard without hand-waving.

1. Overall steps to design the database

Think of the process as a pipeline:

1. Understand the domain and workflows

- **Goal:** Capture how BOSS-style software actually works day to day.
- **For BOSS-like system, identify flows such as:**
 - New commercial customer → contract → scheduled visits → crews run routes → time & materials logged → invoices → payments.
- Write these as short scenarios; you'll use them to test your schema.

2. Identify core entities (nouns)

- **Look for stable nouns** in those scenarios:
 - **Account, Property, Contract, Service, Job, JobTicket, Crew, Route, TimeEntry, MaterialUsage, Invoice, Payment, User, Equipment, InventoryItem**, etc.
- Each of these is a candidate **table**.

3. List attributes for each entity

- For each entity, list the data you need to store:
 - **Account:** Name, BillingAddress, Phone, etc.
 - **JobTicket:** ScheduledDate, Status, Crew, Notes, etc.
- Don't worry about perfect structure yet—just capture everything.

4. Define relationships between entities

- Decide cardinalities:
 - One Account has many Properties.
 - One Contract has many ContractServices.
 - One Job has many JobTickets.
 - One JobTicket has many TimeEntries and MaterialUsages.
- This gives you your **foreign keys**.

5. Choose keys for each table

- **Primary key:** Usually a surrogate like INT IDENTITY in SQL Server.
- **Candidate/natural keys:** AccountNumber, InvoiceNumber, etc.
- Ensure every row is uniquely identifiable.

6. Normalize the design (1NF → 2NF → 3NF)

- Start from your “raw” tables and apply normalization rules (we’ll walk these in detail in a moment).
- This is where you split out repeating data, separate catalogs from transactions, etc.

7. Add constraints and rules

- **NOT NULL, CHECK, UNIQUE, FOREIGN KEY** constraints.
- Example: `JobTicket.JobId` must reference an existing `Job`.

8. Consider performance and denormalization (later)

- Only after you have a clean normalized model.
- Example: add `AccountId` directly on `JobTicket` for reporting, even though it can be reached via `Job → ContractService → Contract → Account`.

2. Normalization rules (1NF, 2NF, 3NF) in plain language

We’ll keep it to the three you’ll actually talk about in an interview.

2.1 First Normal Form (1NF)

Rule (simple):

- Each column holds **atomic** values (no lists, no repeating groups).
- Each row/column intersection has **one value**, not a set.
- No “Material1, Material2, Material3” columns.

How to apply:

- Scan each table:
 - If you see columns like `Material1, Material2, Material3` → that’s a repeating group.
 - If you see a column like `MaterialsUsed = 'Mulch:5,Bags;Salt:10,Bags'` → that’s multiple values in one column.

Fix:

- Create a **child table** for the repeating thing.
 - Example:
 - Bad: `WorkOrder(Material1, Material2, Material3, ...)`
 - Good: `WorkOrder + WorkOrderMaterial(WorkOrderId, InventoryItemId, Quantity)`.

2.2 Second Normal Form (2NF)

2NF only matters when your primary key is **composite** (made of multiple columns).

Rule (simple):

- Every non-key column must depend on the **whole** key, not just part of it.

How to apply:

- Look at tables with composite keys, e.g. (RouteId, StopOrder) or (ContractId, ServiceId).
- Ask: “Does this column depend on both parts of the key, or only one?”

Example:

- Suppose you had a table:

```
ContractService(ContractId, ServiceId, ContractName, ServiceName, PricePerUnit)
```

- Key: (ContractId, ServiceId)
- ContractName depends only on ContractId, not on ServiceId.
- ServiceName depends only on ServiceId, not on ContractId.

Fix:

- Move attributes to the table where they belong:
 - ContractName → Contract table.
 - ServiceName → ServiceCatalog table.
- Keep ContractService(ContractId, ServiceId, PricePerUnit, ...) as the **link**.

If you mostly use surrogate keys (IDENTITY), you’ll hit 2NF issues less often, but the principle is still useful when you design link tables.

2.3 Third Normal Form (3NF)

Rule (simple):

- No **transitive dependencies**: non-key columns must depend **only** on the key, not on other non-key columns.

How to apply:

- For each table, pick a non-key column and ask:
 - “Does this depend directly on the primary key, or does it depend on another non-key column?”

Example:

- Suppose Account has:

```
Account(AccountId, BillingPostal, BillingCity, BillingState, BillingCountry, TaxRate)
```

- If TaxRate is determined by BillingPostal (or BillingState), then TaxRate depends on another non-key column, not directly on AccountId.

Fix:

- Move `TaxRate` to a separate table keyed by the thing it actually depends on, e.g.:

```
TaxRegion(PostalCode, TaxRate)
```

- `Account` then references `TaxRegion` via `BillingPostal` or a `TaxRegionId`.

In practice, for BOSS-like systems, 3NF often means:

- Service descriptions live in `ServiceCatalog`, not in `ContractService`.
 - Account addresses live in `Account`, not repeated on `JobTicket` and `Invoice` (you might copy them for history, but that's a conscious denormalization).
-

3. Applying normalization step-by-step to the BOSS-style DB

Let's take a **concrete example**: imagine you start with a single, ugly `WorkOrder` table someone hacked together.

3.1 Start with a denormalized “WorkOrder” table

Imagine this table:

```
WorkOrder(  
    WorkOrderId,  
    AccountName,  
    AccountBillingAddress,  
    PropertyName,  
    PropertyAddress,  
    ContractNumber,  
    ServiceName,  
    ScheduledDate,  
    CrewName,  
    CrewMember1,  
    CrewMember2,  
    CrewMember3,  
    Material1Name,  
    Material1Qty,  
    Material2Name,  
    Material2Qty,  
    InvoiceNumber,  
    InvoiceTotal  
)
```

This is typical of a first attempt: everything in one place.

3.2 Apply 1NF to this WorkOrder

Find repeating groups and multi-valued columns:

- `CrewMember1`, `CrewMember2`, `CrewMember3` → repeating group.
- `Material1Name`, `Material1Qty`, `Material2Name`, `Material2Qty` → repeating group.

Fix crew members:

- Create Crew and CrewMember tables:

```
Crew(CrewId, Name, ...)
CrewMember(CrewMemberId, CrewId, UserId, Role, ...)
```

- In WorkOrder, keep only CrewId.

Fix materials:

- Create InventoryItem and WorkOrderMaterial:

```
InventoryItem(InventoryItemId, Name, UnitOfMeasure, ...)
WorkOrderMaterial(WorkOrderId, InventoryItemId, QuantityUsed)
```

- Remove Material1* and Material2* columns from WorkOrder.

Now WorkOrder has **no repeating groups**—1NF satisfied.

3.3 Apply 2NF (if needed)

Check for composite keys. Suppose we decide:

- WorkOrderMaterial has primary key (WorkOrderId, InventoryItemId).

Check non-key columns:

- QuantityUsed depends on both WorkOrderId and InventoryItemId → OK.
- If we had InventoryItemName in WorkOrderMaterial, that would depend only on InventoryItemId → 2NF violation.

Fix:

- Move InventoryItemName to InventoryItem table.
- Keep only InventoryItemId in WorkOrderMaterial.

Do the same for any other link tables, like ContractService(ContractId, ServiceId, PricePerUnit).

3.4 Apply 3NF to the same area

Look at WorkOrder again (after 1NF fixes). Maybe it now looks like:

```
WorkOrder(
    WorkOrderId,
    AccountName,
    AccountBillingAddress,
    PropertyName,
    PropertyAddress,
    ContractNumber,
    ServiceName,
    ScheduledDate,
    CrewId,
    InvoiceNumber,
    InvoiceTotal
)
```

Ask: “Which of these columns really describe the WorkOrder itself, and which describe other

entities?”

- AccountName, AccountBillingAddress → describe **Account**, not WorkOrder.
- PropertyName, PropertyAddress → describe **Property**.
- ContractNumber → describes **Contract**.
- ServiceName → describes **Service**.
- InvoiceNumber, InvoiceTotal → describe **Invoice**.

These are **transitive dependencies**: they depend on WorkOrder only through some other entity.

Fix:

1. Create separate tables:

```
Account(AccountId, AccountName, BillingAddress, ...)
Property(PropertyId, AccountId, PropertyName, PropertyAddress, ...)
Contract(ContractId, AccountId, ContractNumber, ...)
ServiceCatalog(ServiceId, ServiceName, ...)
Invoice(InvoiceId, AccountId, InvoiceNumber, InvoiceTotal, ...)
```

2. Replace descriptive columns in **WorkOrder** with foreign keys:

```
WorkOrder(
    WorkOrderId,
    PropertyId,
    ContractId,
    ServiceId,
    ScheduledDate,
    CrewId,
    InvoiceId
)
```

Now:

- Every non-key column in **WorkOrder** depends directly on **WorkOrderId**.
- Account, Property, Contract, Service, Invoice each have their own tables with their own attributes.
- That’s 3NF.

3.5 Map this back to the earlier BOSS-style schema

What we just did is exactly how we arrived at entities like:

- Account, Property, Contract, ServiceCatalog
- Job / JobTicket (instead of a single WorkOrder table)
- TicketMaterialUsage instead of Material1, Material2, ...
- Crew + CrewMember instead of CrewMember1, CrewMember2, ...
- Invoice + InvoiceLine instead of invoice data on the work order

You can literally tell the story: “I started from a flat WorkOrder table and normalized it into these

entities.”

4. How to talk through this in an Integra interview

If they say “How would you design/normalize our database?” you can walk them through:

1. Domain understanding:

- “First I’d map your workflows: contracts, work orders, routes, job costing, billing.”

2. Entity identification:

- “From that I’d identify entities like Account, Property, Contract, Job, JobTicket, Crew, TimeEntry, MaterialUsage, Invoice, Payment.”

3. Normalization steps:

- “I’d start with a naive WorkOrder table and then:
 - Apply **1NF** to remove repeating groups (crew members, materials).
 - Apply **2NF** to ensure link tables only store attributes that depend on the full composite key.
 - Apply **3NF** to move account, property, contract, and service details into their own tables, leaving WorkOrder/JobTicket with only attributes that depend on its key.”

4. Resulting structure:

- “That leads to a clean, normalized core: Account → Property → Contract → Job → JobTicket → TimeEntry/MaterialUsage → Invoice/Payment.”

If you want, next we can take **one specific area** (e.g., Contracts & JobTickets) and do a full “from messy spreadsheet → 3NF tables” walkthrough with actual SQL Server DDL so you can rehearse it.